Proyecto final DAW

PDAW

DAW Desarrollo de Aplicaciones Web

19-3-2026

Indice

Descripción del Proyecto....6
Contexto Actual....6
Estructura del Grupo....6
Ejemplo actual de uso para incorporar una familia:....6
Ejemplo actual de uso semanal para una familia que quiere pedir verduras:....6
Ejemplo actual de uso semanal para una familia que quiere pedir huevos:....6
Ejemplo actual de uso semanal para la familia que gestiona las verduras:....7
Ejemplo actual de uso semanal para la familia que gestiona el pan (periódico):....7
Pedidos....7
Pedido abierto....7
Pedidos periódicos....7
Frecuencia de Pedidos....8
Entrega de Productos....8
Reparto de Productos....8
Gestión Actual en Google Sheets....8
Cálculos Avanzados....8
Propuesta de Solución....8
Herramientas propuestas....9
Tecnologías de Programación....9
Backend API Rest....9
Frontend Web....10
Arquitectura del Sistema....10
Componentes:....10
Diagrama:....10
Flujo de Usuario....10
Resumen de flujo de usuario:....11
Diagrama:....11
Autenticación y Autorización con JWT....11
¿Qué es JWT?....11
Flujo de Autenticación con JWT:....11
Estructura del Token JWT:....12
Implementación en el Proyecto:....12
Ventajas de JWT:....13
Rutas Protegidas....13
Datos....15
Diagrama de Base de Datos (ER)....15
Tablas Principales:....15
Diagrama:....15
Tablas Principales....16
1. Usuario....16
2. Proveedor....16
3. Producto....16
4. Pedido....17
5. Detalle_Pedido....17
6. Familia_Proveedor (Relación muchos a muchos)....17
7. Pedido_Periodico....17

8. Pago....18
9. Notificacion....18
Pedido_Periodico_Generacion....994
Backend API Rest Node Express....18
Descripción del Backend....18
Estructura del Proyecto....18
Requisitos del Backend....19
1. Autenticación y Autorización....19
2. Gestión de Productos....19
3. Gestión de Pedidos....19
4. Historial de Pedidos....20
5. Configuración de Cuenta....20
6. Notificaciones....20
7. Documentación de la API....20
Ejemplo de Rutas de la API....20
Test....20
Usuarios....20
Productos....21
Proveedores....21
Pedidos....21
Detalle de Pedido....21
Familia-Proveedor....22
Pedidos Periódicos....22
Pagos....22
Notificaciones....23
Frontend Web VUE.js....23
Descripción del Frontend....23
Estructura del Proyecto....23
Diseño Visual....24
Guía de Estilo....24
Paleta de Colores....24
Tipografía....24
Iconografía....24
Elementos de Interfaz....24
Botones....24
Formularios....25
Tarjetas de Producto....25
Prototipos de la Aplicación Web....25
1. Pantalla de Inicio....25
2. Pantalla de Registro....26
3. Pantalla de Inicio de Sesión....26
4. Pantalla de Recuperar Contraseña....26
5. Pantalla de Dashboard (Tablero)....27
6. Pantalla de Compras....27
7. Pantalla de Historial de Pedidos....27
8. Pantalla de Detalles de Pedidos....28
9. Pantalla de Gestión de Usuarios (para administradores)....28
10. Pantalla de Configuración de Cuenta....28

11. Pantalla de Soporte y Ayuda....29
12. Pantalla de Gestión de Proveedores....29
13. Pantalla de Gestión de Productos....29
14. Pantalla de Gestión de Pedidos....30
15. Barra de navegación....30
16. Pie de página....31
Flujo de Usuario para la Aplicación Web....31
1. Gestión de Productos....31
2. Gestión de Compras....31
2. Gestión de Pedidos....32
3. Cálculos Financieros (para administradores)....33
4. Gestión de Usuarios (para administradores)....33
5. Notificaciones y Recordatorios....34
6. Historial de Pedidos....34
7. Configuración de Cuenta....34
8. Panel de Administración (para administradores)....35
9. Informes y Estadísticas....35
Diagrama de Flujo de Pedidos....36
Pasos:....36
Diagrama:....36
Diagrama de Navegación....36
Páginas Principales:....36
Diagrama de Secuencia de Autenticación....37
Pasos:....37
Diagrama:....37
Diagrama de Componentes reutilizables del Frontend....37
Componentes:....37
Diagrama:....37
Entorno de Producción....39
Clouding.io....39
Scripts para Instalar el Entorno y Arrancar el Proyecto....39
DuckDNS.org....39
Subida o Actualización de Código....39
Seguridad....39
Copias de Seguridad....40
Monitoreo....40

Fecha	Versión	Autor	Descripción

28/11/2025	**0.2**	**Eneko Bladi**	**Correcciones para segunda entrega**
19/03/2026	**0.3**	**Eneko Bladi**	**Correcciones para segunda entrega**
27/03/2026	**0.4**	**Eneko Bladi**	**Correcciones: actualización de nombres de archivos, estructuras y tecnologías para reflejar código actual.**

Descripción del Proyecto

El objetivo de este proyecto es desarrollar una aplicación web que facilite la gestión de un grupo de consumo local y ecológico, compuesto por aproximadamente cuarenta usuarios (familias) y quince productores. Actualmente, la gestión de pedidos y productos se realiza de manera manual a través de tablas en Google Sheets, lo que puede resultar ineficiente y propenso a errores.

Contexto Actual

Estructura del Grupo

El grupo está formado por familias que se encargan de contactar directamente con los productores. Cada usuario asume la responsabilidad de mantenerse en contacto con un proveedor específico, aunque hay algunos usuarios que no gestionan ningún proveedor. Será el usuario que gestiona ese proveedor el encargado de dar de alta, modificar y borrar productos. Será el encargado de dar de alta pedidos conjuntos estableciendo un periodo en el que los usuarios podrán añadir lineas al pedido.

Ejemplo actual de uso para incorporar una familia:

1. Contacta con responsable del grupo de consumo.
2. Responsable explica funcionamiento de grupo.
3. Responsable pide usuario de Google a la familia.
4. Responsable comparte la hoja de cálculo de Google con la familia.

Ejemplo actual de uso semanal para una familia que quiere pedir verduras:

1. Entra a Google Docs.
2. Entra a la sheet compartida.
3. Busca la hoja correspondiente al productor de verduras (semanal).
4. Comprueba que el cuadro corresponde a la semana actual (se pone cabecera verde).
5. Se sitúa en la columna correspondiente a su familia.
6. Va introduciendo las unidades de cada producto que desea.
7. En la parte de abajo le calcula el importe total.
8. La semana siguiente tendrá la nueva tabla vacía.

Ejemplo actual de uso semanal para una familia que quiere pedir huevos:

1. Entra a Google Docs.
2. Entra a la sheet compartida.
3. Busca la hoja correspondiente al productor de huevos (semanal y periódico).
4. Comprueba que el cuadro corresponde a la semana actual (se pone cabecera verde).
5. Se sitúa en la columna correspondiente a su familia.
6. Introduce las unidades.
7. En la parte de abajo le calcula el importe total.

8. Hasta que modifique las unidades, recibirá ese pedido semanalmente de manera automática.

Ejemplo actual de uso semanal para la familia que gestiona las verduras:

1. Entra a Google Docs.
2. Entra a la sheet compartida.
3. Busca la hoja correspondiente al productor de verduras (semanal).
4. Busca el cuadro con la suma total de unidades de cada verdura.
5. Contacta con el proveedor o automáticamente se le envía correo desde la sheet.
6. (Esto ahora es automático) Duplica el cuadro (vacío) de la semana pasada en las filas inferiores y se establece la cabecera del pedido actual en verde y la anterior en rojo.
7. Al final de mes, el productor le pasa factura.
8. Comprueba que el importe coincide con los pedidos.

Ejemplo actual de uso semanal para la familia que gestiona el pan (periódico):

1. Entra a Google Docs.
2. Entra a la sheet compartida.
3. Busca la hoja correspondiente al productor de pan (semanal).
4. Busca el cuadro con la suma total de unidades de cada pan.
5. Contacta con el proveedor o automáticamente se le envía correo desde la sheet.
6. (Esto ahora es automático) Duplica el cuadro de la semana pasada (arrastrando las unidades también) en las filas inferiores y se establece la cabecera del pedido actual en verde y la anterior en rojo.
7. Al final de mes, el productor le pasa factura.
8. Comprueba que el importe coincide con los pedidos.

Pedidos

Los pedidos se hacen de manera conjunta entre todos los usuarios del grupo. Los pedidos los abre el usuario encargado de ese proveedor(grupo de productos).

Pedido abierto

Cuando un pedido está abierto significa que el usuario encargado del proveedor ha abierto un pedido y ha establecido una fecha de apertura y una fecha de cierre. El pedido solo será editable en este periodo.

Pedidos periódicos

Hay algunos productos que se piden semanalmente de manera automática sin que tenga que intervenir el usuario hasta que quiera dejar de recibir ese pedido.

Frecuencia de Pedidos

Los productos se solicitan a diferentes proveedores con distintas frecuencias: algunos se piden semanalmente, mientras que otros se solicitan cada dos, tres o seis meses.

Entrega de Productos

Todos los productos se entregan en un local común, donde los usuarios pueden recoger sus pedidos. Es el usuario encargado de ese proveedor el que establecerá una fecha aproximada de entrega en el momento de abrir un pedido.

Reparto de Productos

Algunos productos se repartirán a los usuarios que los han pedido, después de que hayan sido entregados en el local.

Gestión Actual en Google Sheets

La gestión de pedidos y productos se organiza en Google Sheets de la siguiente manera:

- **Páginas por Proveedor**: Cada proveedor tiene su propia página en la hoja de cálculo.
- **Registro de Productos**: Cada producto se representa en una fila, lo que permite un seguimiento claro de la disponibilidad y características de cada uno.
- **Pedidos**: Cada pedido se compone de varias filas, donde se establece una fecha máxima de modificación, la fecha de pedido y la periodicidad de los mismos.
- **Usuarios y Familias**: Cada familia o usuario tiene su propia columna, lo que facilita el seguimiento de sus pedidos y gastos.

Cálculos Avanzados

Se realizan cálculos complejos para gestionar la información financiera y de pedidos:

- **Gasto Mensual**: Se calcula el gasto mensual de cada familia. Aquellos que gestionan un proveedor y realizan el pago de los pedidos tienen un saldo positivo. Al final del mes, se determina qué familias deben dinero y a quién se lo deben.
- **Cálculo de Pedidos**: Se establece un cálculo para determinar el pedido que debe enviarse al proveedor o productor antes de la fecha límite.

Propuesta de Solución

La nueva aplicación web permitirá automatizar y optimizar todos estos procesos, ofreciendo una interfaz más amigable y eficiente para los usuarios. Algunas de las características clave incluirán:

- **Gestión de Productos y Proveedores**: Una sección dedicada para que los usuarios gestores puedan ver y gestionar los productos y proveedores de manera más intuitiva.
- **Sistema de Pedidos**: Un sistema que permita abrir pedidos de forma sencilla a los usuarios gestores.

- **Sistema de Compras**: Un sistema que permita añadir productos a la cesta mensual de manera intuitiva.
- **Cálculos Automáticos**: Automatización de los cálculos de gastos y deudas, eliminando la necesidad de realizar cálculos manuales en hojas de cálculo.
- **Interacción entre Usuarios**: Facilitar la comunicación entre usuarios y productores, mejorando la colaboración y la eficiencia del grupo.

Herramientas propuestas

- **GitHub**: Para el control de versiones y el repositorio remoto.
- **Visual Studio Code**: Editor de código muy versátil, con mucha comunidad, e infinidad de extensiones:
    - **Git Graph**: Para visualizar ramas y commits de Git de manera gráfica.
    - **Debugger for Firefox**: Para depurar el código directamente en Firefox.
    - **Vue**: La extensión oficial de Vue, con autocompletado y herramientas de depuración.
- **Penpot**: Lo he usado para prototipar interfaces. Es gratis y muy intuitivo.
- **Draw.io**: Para crear diagramas rápidos, como flujos de trabajo o esquemas de la base de datos.
- **Gimp**: Editor de imágenes para retocar o optimizar imágenes para la app.
- **Clouding.io**: VPS que he configurado con - para alojar la app en producción. Lo he elegido porque no es caro y es una empresa relativamente de cercanía.
- **Duckdns.org**: Para tener un nombre de dominio gratuito para acceder a la web sin conocer la IP.
- **VUE dev tools**: Estensión de navegador para depurar aplicaciones desarrolladas con VUE.

Tecnologías de Programación

- **Node.js**: Entorno de ejecución JavaScript que permite usar el mismo lenguaje en frontend y backend.
- **npm**: Gestor de paquetes de Node.js. Permite instalar paquetes y librerías y gestionar dependencias de manera sencilla.

Backend API Rest

- **PostgreSQL**: El motor de base de datos elegido. Es robusto y escalable, ideal para manejar datos complejos. Nunca lo había probado y me apetecía.
- **Swagger**: Herramienta para documentar la API de manera interactiva. Así los desarrolladores frontend pueden entender cómo usarla. Además incluye una web para probar los endoints y nos ahorramos utilizar herramientas como Postman.
- **Express**: Framework en Node para crear la API. Es flexible y fácil de usar.
- **Cors**: Para permitir que el frontend se comunique con el backend sin problemas de CORS.
- **express-validator**: Para validar y sanitizar los datos que llegan al backend.

- **swagger-jsdoc**: Genera la documentación de Swagger automáticamente desde los comentarios del código.
- **swagger-ui-express**: Sirve la interfaz de Swagger para probar la API desde el navegador.
- **dotenv**: Para gestionar variables de entorno, como credenciales de la base de datos.
- **multer**: Para manejar la subida de archivos, como las imágenes de los productos.
- **nodemailer**: Permite enviar emails mediante diferentes servicios de correo.
- **bcryptjs**: Permite almacenar las contraseñas de forma segura mediante hashing.
- **jsonwebtoken (JWT)**: Permite generar y verificar tokens JWT (JSON Web Tokens) para implementar autenticación y autorización seguras.
- **Jest**: Framework de pruebas para JavaScript.
- **Supertest**: Herramienta para realizar pruebas de endpoints HTTP.

Frontend Web

- **Vue.js**: Framework de Node para el frontend. Es sencillo y con un desarrollo ágil. Igual que casi todo el resto de tecnologías mencionadas quiero probar una tecnología que desconozco.
- **vue-router**: Para gestionar la navegación entre vistas en una aplicación de una sola página (SPA).
- **Bootstrap**: Para crear interfaces modernas y responsivas sin complicaciones.
- **axios**: Para hacer peticiones HTTP a la API desde el frontend.
- **Pinia**: Para gestionar el estado global de la aplicación de manera centralizada.
- **Localstorage**: Para guardar los datos en sesiones etc...

Arquitectura del Sistema

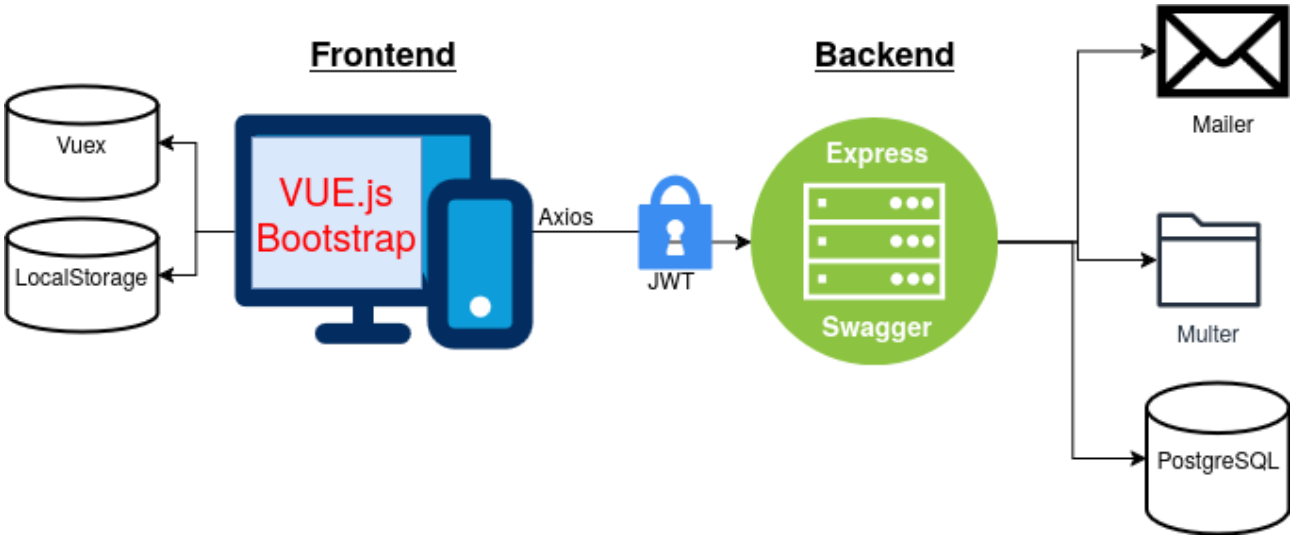
Este diagrama muestra los componentes principales del sistema y cómo interactúan entre sí.

Componentes:

- **Frontend (Node.js + Vue.js + Bootstrap)**: Interfaz de usuario para los usuarios del grupo de consumo.
- **Backend (Node.js + Express)**: API REST que gestiona las solicitudes del frontend.
- **Base de Datos (PostgreSQL)**: Almacena la información de usuarios, productos, proveedores, pedidos, etc.
- **Almacenamiento de ficheros. En este caso imágenes (Multer)**
- **Servicios Externos**: Notificaciones por correo electrónico o SMS. Librearias Node.js

Diagrama:

Flujo de Usuario



Este diagrama representa el flujo de un usuario típico en la aplicación.

Resumen de flujo de usuario:

1. **Inicio de Sesión/Registro**: El usuario inicia sesión o se registra.
2. **Dashboard**: El usuario accede a su panel principal.
3. **Gestión de Productos**: El usuario gestor añade, edita o elimina productos.
4. **Gestión de Pedidos**: El usuario gestor abre o cierra pedidos.
5. **Historial de Pedidos**: El usuario consulta su historial de pedidos.
6. **Configuración de Cuenta**: El usuario actualiza su información personal.

Diagrama:

```
[Inicio de Sesión/Registro] -> [Dashboard]  -> [Gestión de Productos]
                                            -> [Gestión de Pedidos]
                                            -> [Historial de Pedidos]
                                            -> [Configuración de Cuenta]
```

Autenticación y Autorización con JWT

¿Qué es JWT?

JSON Web Tokens (JWT) es un estándar para representar datos de forma segura entre dos partes. Lo utilizo para autenticar usuarios y proteger rutas sensibles en la API.

Flujo de Autenticación con JWT:

1. **Inicio de Sesión**:
    - El usuario envía sus credenciales (correo/móvil y contraseña) al backend.
    - El backend valida las credenciales y genera un token JWT si son correctas.

- El token se devuelve al frontend y se almacena en el `localStorage`
- El token es valido durante 30 minutos desde que se genera.

2. **Acceso a Rutas Protegidas**:
   - El frontend incluye el token en el encabezado `Authorization` de cada solicitud HTTP.
   - El backend verifica el token antes de permitir el acceso a las rutas protegidas.
3. **Cierre de Sesión**:
   - El frontend elimina el token del almacenamiento local.
   - El usuario ya no puede acceder a rutas protegidas.

Estructura del Token JWT:

Un token JWT consta de tres partes:

1. **Header**: Contiene el tipo de token y el algoritmo de firma.
2. **Payload**: Contiene los datos del usuario (como `id`, `correo`, `rol`) y la fecha de expiración.
3. **Signature**: Garantiza que el token no ha sido alterado.

Ejemplo de un token JWT:


```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwiY29ycmVvIjoiam9obkBleGFtcGxlLmN
vbSIsInJvbCI6InVzZXIiLCJpYXQiOjE2MjYyNzYwMDAsImV4cCI6MTYyNjI3OTYwMH0.4f5c8b1d8e9
e8f8f8f8f8f8f8f8f8f8f8f8f8f8
```


Implementación en el Proyecto:

1. **Generación del Token**:
   - Se utiliza la biblioteca `jsonwebtoken` para generar el token en el backend.
   - Ejemplo:

```
const jwt = require('jsonwebtoken');
const SECRET_KEY = 'clave_secreta';

const token = jwt.sign(
  { id: usuario.id, correo: usuario.correo, rol: usuario.rol },
  SECRET_KEY,
  { expiresIn: '1h' } // El token expira en 1 hora
);
```

2. **Verificación del Token**:
   - Se utiliza un middleware para verificar el token en las rutas protegidas.
   - Ejemplo:

```
const authMiddleware = (req, res, next) => {
  const token = req.headers['authorization'];
  if (!token) {
    return res.status(401).send('Acceso denegado. No se proporcionó
un token.');
  }
```

```
  try {
    const decoded = jwt.verify(token, SECRET_KEY);
    req.user = decoded; // Agrega los datos del usuario al objeto
`req`
    next();
  } catch (err) {
    res.status(401).send('Token inválido o expirado.');
  }
};
```

3. **Uso en el Frontend**:
   - El token se almacena en el `localStorage` después de iniciar sesión:

     ```
     localStorage.setItem('authToken', token);
     ```

   - Se incluye en las solicitudes HTTP usando Axios:

     ```
     apiClient.interceptors.request.use((config) => {
       const token = localStorage.getItem('authToken');
       if (token) {
         config.headers['Authorization'] = token;
       }
       return config;
     });
     ```

Ventajas de JWT:

- **Seguridad**: Los datos están firmados y no pueden ser alterados.
- **Escalabilidad**: No requiere almacenar sesiones en el servidor.
- **Flexibilidad**: Puede incluir información adicional en el payload.

Rutas Protegidas

Se van a proteger todas las rutas a excepción de aquellas en las que el usuario no ha podido obtener el token.

Las rutas que no están protegidas:

- Prueba la conexión con la API y la base de datos.
- Registra un nuevo usuario.
- Autentica un usuario (login).
- Envía un enlace para recuperar la contraseña.

Las siguientes rutas están protegidas y requieren un token JWT válido para acceder. Además, como en el token tenemos el rol del usuario logueado, hay endpoints que solo son accesibles para administradores:

- Obtiene todos los usuarios (solo administradores).
- Obtiene un usuario por su ID.

- Activa o desactiva un usuario (solo administradores).
- Actualiza un usuario.
- Elimina un usuario (solo administradores).
- Obtiene el saldo de un usuario.
- Crea un nuevo producto.
- Obtiene todos los productos.
- Obtiene un producto por su ID.
- Actualiza un producto.
- Elimina un producto.
- Cambia el estado activo de un producto.
- Crea un nuevo proveedor.
- Obtiene todos los proveedores.
- Obtiene un proveedor por su ID.
- Cambia el estado activo de un proveedor.
- Crea un nuevo pedido.
- Obtiene todos los pedidos.
- Obtiene un pedido por su ID.
- Actualiza un pedido.
- Elimina un pedido.
- Cambia el estado de un pedido.
- Cambia el estado activo de un pedido.
- Crea un nuevo detalle de pedido.
- Obtiene todos los detalles de un pedido.
- Actualiza un detalle de pedido.
- Elimina un detalle de pedido.
- Crea una nueva relación entre usuario y proveedor.
- Obtiene todas las relaciones de un usuario.
- Obtiene todas las relaciones de un proveedor.
- Elimina una relación entre usuario y proveedor.
- Crea un nuevo pedido periódico.
- Obtiene todos los pedidos periódicos.
- Obtiene los pedidos periódicos de un usuario.
- Actualiza un pedido periódico.
- Elimina un pedido periódico.
- Cambia el estado activo de un pedido periódico.
- Crea un nuevo pago.
- Obtiene todos los pagos.
- Obtiene los pagos pendientes de un usuario deudor.
- Obtiene los pagos pendientes de un usuario acreedor.
- Cambia el estado de un pago.
- Envía una notificación.
- Marca una notificación como leída.

Datos

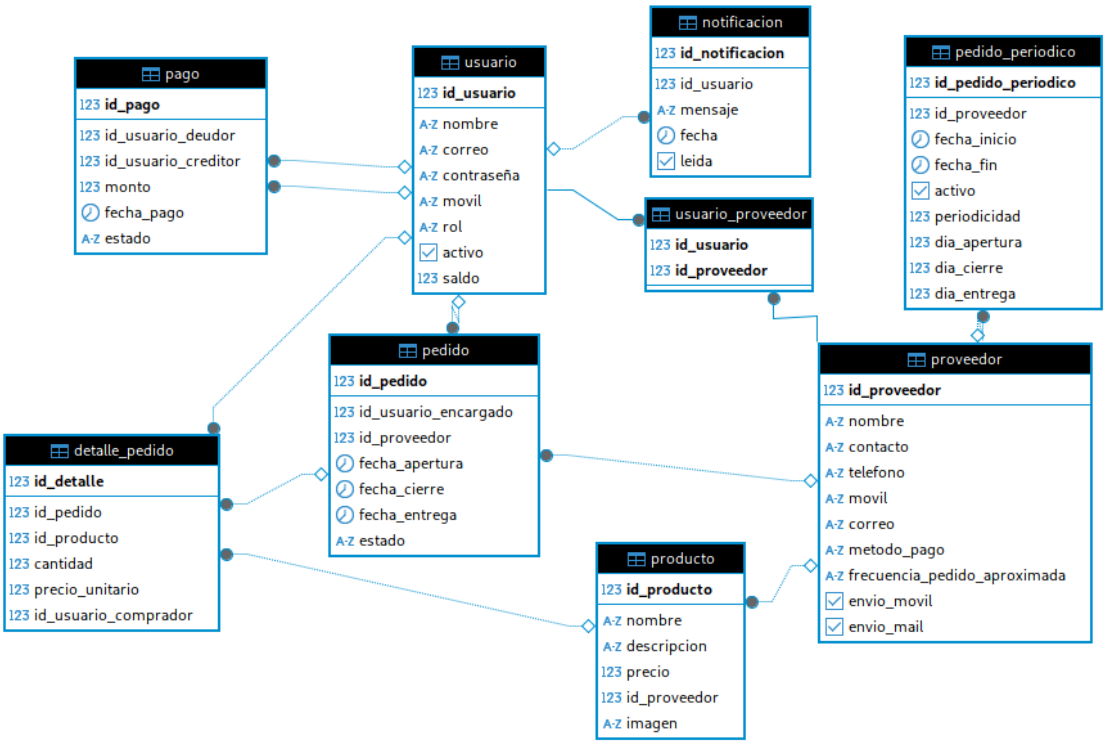
Diagrama de Base de Datos (ER)

Este diagrama muestra las tablas de la base de datos y sus relaciones.

Tablas Principales:

- **Usuario**: Almacena la información de los usuarios.
- **Proveedor**: Almacena la información de los proveedores.
- **Producto**: Almacena la información de los productos.
- **Pedido**: Almacena la información de los pedidos.
- **Detalle_Pedido**: Relaciona los pedidos con los productos.
- **Usuario_Proveedor**: Relación muchos a muchos entre usuarios y proveedores.
- **Pedido_Periodico**: Almacena la información de los pedidos periódicos.
- **Pago**: Almacena la información de los pagos.
- **Notificacion**: Almacena las notificaciones enviadas a los usuarios.

Diagrama:



Tablas Principales

1. Usuario

Columna	Tipo de Dato	Descripción
`ID_Usuario`	`SERIAL`	Clave primaria.
`Nombre`	`VARCHAR(100)`	Nombre del usuario.
`Correo`	`VARCHAR(100)`	Correo electrónico (único).
`Contraseña`	`VARCHAR(100)`	Contraseña del usuario.
`Movil`	`VARCHAR(20)`	Teléfono móvil del usuario.
`Rol`	`VARCHAR(50)`	Rol del usuario (admin, gestor, usuario).
`Activo`	`BOOLEAN`	Indica si el usuario está activo.
`Saldo`	`DECIMAL(10, 2)`	El saldo se calcula a partir de los pagos pendientes (no es un campo almacenado).
`Fecha_Modificacion`	`TIMESTAMP`	Fecha y hora de modificación del usuario.

---

2. Proveedor

Columna	Tipo de Dato	Descripción
`ID_Proveedor`	`SERIAL`	Clave primaria.
`Nombre`	`VARCHAR(100)`	Nombre del proveedor/grupo productos
`Contacto`	`VARCHAR(100)`	Nombre de la persona de contacto.
`Telefono`	`VARCHAR(20)`	Teléfono del proveedor.
`Movil`	`VARCHAR(20)`	Teléfono móvil del proveedor.
`Correo`	`VARCHAR(100)`	Correo electrónico del proveedor.
`Metodo_Pago`	`VARCHAR(300)`	Como se paga al proveedor
`Envio_Movil`	`BOOLEAN`	Avisar mediante SMS o WhatsApp.
`Envio_Mail`	`BOOLEAN`	Avisar mediante correo electrónico.
`Fecha_Modificacion`	`TIMESTAMP`	Fecha y hora de modificación del proveedor.

---

3. Producto

Columna	Tipo de Dato	Descripción
`ID_Producto`	`SERIAL`	Clave primaria.
`Nombre`	`VARCHAR(100)`	Nombre del producto.
`Imagen`	`VARCHAR(100)`	URL relativa de imagen del producto.
`Descripcion`	`TEXT`	Descripción del producto.
`Precio`	`DECIMAL(10, 2)`	Precio del producto.
`Frecuencia_Pedido`	`VARCHAR(50)`	Este campo ya no se utiliza (se eliminó de la implementación).
`ID_Proveedor`	`INT`	Clave foránea (relación con `Proveedor`).

Columna	Tipo de Dato	Descripción
Fecha_Modificacion	TIMESTAMP	Fecha y hora de modificación del proveedor.

4. Pedido

Columna	Tipo de Dato	Descripción
ID_Pedido	SERIAL	Clave primaria.
ID_Usuario_Encargado	INT	Clave foránea (relación con Usuario).
ID_Proveedor	INT	Clave foránea (relación con Proveedor).
Fecha_Modificacion	TIMESTAMP	Fecha y hora de modificación del pedido.
Fecha_Apertura	TIMESTAMP	Fecha y hora de apertura del pedido.
Fecha_Cierre	TIMESTAMP	Fecha límite para modificar el pedido.
Fecha_Entrega	TIMESTAMP	Fecha y hora de entrega.
Estado	VARCHAR(50)	Estado del pedido (pendiente, en proceso, entregado, repartido, cancelado).

5. Detalle_Pedido

Columna	Tipo de Dato	Descripción
ID_Detalle	SERIAL	Clave primaria.
ID_Pedido	INT	Clave foránea (relación con Pedido).
ID_Producto	INT	Clave foránea (relación con Producto).
Cantidad	INT	Cantidad del producto.
Precio_Total	DECIMAL(10, 2)	Precio total del producto.
ID_Usuario_Comprador	INT	Clave foránea (relación con Usuario).
Fecha_Modificacion	TIMESTAMP	Fecha y hora de modificación del detalle.

6. Usuario_Proveedor (Relación muchos a muchos)

Columna	Tipo de Dato	Descripción
ID_Usuario	INT	Clave foránea (relación con Usuario).
ID_Proveedor	INT	Clave foránea (relación con Proveedor).

7. Pedido_Periodico

Columna	Tipo de Dato	Descripción
ID_Pedido_Periodico	SERIAL	Clave primaria.
ID_Proveedor	INT	Clave foránea (relación con Proveedor).
Fecha_Inicio	TIMESTAMP	Fecha de inicio del pedido periódico.

Columna	Tipo de Dato	Descripción
`Fecha_Fin`	`TIMESTAMP`	Fecha de fin del pedido periódico.
`Activo`	`BOOLEAN`	Indica si el pedido está activo.
`Periodicidad`	`INT`	Días entre pedidos.
`Dia_Apertura`	`INT`	Día de la semana en que se abre el pedido.
`Dia_Cierre`	`INT`	Día de la semana en que se cierra el pedido.
`Dia_Entrega`	`INT`	Día aproximado de entrega.

Generación Automática

Los pedidos periódicos se generan automáticamente mediante un job cron que ejecuta el script generarPedidosPeriodicos.js. Este script:

- Lee las configuraciones activas de Pedido_Periodico
- Calcula si corresponde crear un pedido según la periodicidad
- Crea el pedido en la base de datos con fechas de apertura, cierre y entrega
- Registra la generación en la tabla Pedido_Periodico_Generacion

---

8. Pago

Columna	Tipo de Dato	Descripción
`ID_Pago`	`SERIAL`	Clave primaria.
`ID_Usuario_Deudor`	`INT`	Clave foránea (relación con `Usuario`).
`ID_Usuario_Creditor`	`INT`	Clave foránea (relación con `Usuario`).
`Monto`	`DECIMAL(10, 2)`	Monto del pago.
`Fecha_Pago`	`TIMESTAMP`	Fecha y hora del pago.
`Estado`	`VARCHAR(50)`	Estado del pago (pendiente, completado).
`Fecha_Modificacion`	`TIMESTAMP`	Fecha y hora de modificacion del pago.

---

9. Notificacion

Columna	Tipo de Dato	Descripción
`ID_Notificacion`	`SERIAL`	Clave primaria.
`ID_Usuario`	`INT`	Clave foránea (relación con `Usuario`).
`Mensaje`	`TEXT`	Contenido de la notificación.
`Fecha`	`TIMESTAMP`	Fecha y hora de la notificación.
`Leida`	`BOOLEAN`	Indica si la notificación ha sido leída.
`Fecha_Modificacion`	`TIMESTAMP`	Fecha y hora de modificación de la notifiacion.

---

10. Pedido_Periodico_Generacion

Columna	Tipo de Dato	Descripción
ID_Pedido_Periodico_Generacion	SERIAL	Clave primaria.
ID_Pedido_Periodico	INT	Clave foránea (relación con Pedido_Periodico).
Fecha_Ultima_Generacion	TIMESTAMP	Fecha y hora de la última generación del pedido.

Backend API Rest Node Express

Descripción del Backend

El backend de la aplicación está desarrollado con **Node.js** y **Express**. Se conecta a una base de datos **PostgreSQL** y expone una API REST para gestionar usuarios, productos, proveedores y pedidos. La aplicación está diseñada para ser escalable y segura.

Estructura del Proyecto

El backend está estructurado de la siguiente manera:

```
backend/
├── src/
│   ├── controllers/        # Controladores para manejar las solicitudes
│   │   ├── UsuarioController.js          # Controlador de usuarios y autenticación
│   │   ├── UsuarioController.js          # Controlador de usuarios
│   │   ├── ProductoController.js       # Controlador de productos
│   │   ├── ProveedorController.js      # Controlador de proveedores
│   │   ├── PedidoController.js         # Controlador de pedidos
│   │   ├── PedidoPeriodicoController.js # Controlador de pedidos periódicos
│   │   ├── PagoController.js       # Controlador de pagos
│   │   ├── NotificacionController.js  # Controlador de notificaciones
│   │   └── DetallePedidoController.js   # Controlador de detalles de pedidos
│   ├── models/             # Modelos de datos
│   │   ├── Usuario.js            # Modelo de usuario
│   │   ├── Producto.js         # Modelo de producto
│   │   ├── Proveedor.js        # Modelo de proveedor
│   │   ├── Pedido.js           # Modelo de pedido
│   │   ├── PedidoPeriodico.js   # Modelo de pedidos periódicos
│   │   ├── Pago.js         # Modelo de pagos
│   │   ├── Notificacion.js    # Modelo de notificaciones
│   │   └── DetallePedido.js     # Modelo de detalles de pedidos
│   ├── routes/             # Rutas de la API
│   │   ├── UsuarioRoutes.js           # Rutas de autenticación
│   │   ├── UsuarioRoutes.js           # Rutas de usuarios
│   │   ├── ProductoRoutes.js        # Rutas de productos
│   │   ├── ProveedorRoutes.js       # Rutas de proveedores
│   │   ├── PedidoRoutes.js          # Rutas de pedidos
│   │   ├── PedidoPeriodicoRoutes.js  # Rutas de pedidos periódicos
│   │   ├── PagoRoutes.js        # Rutas de pagos
│   │   └── NotificacionRoutes.js   # Rutas de notificaciones
│   ├── config/             # Configuración de la aplicación
│   │   ├── dbConfig.js            # Configuración de la base de datos
│   │   ├── serverConfig.js        # Configuración del servidor
│   │   ├── multerConfig.js        # Configuración de Multer para subir archivos
│   │   └── swagger.js       # Configuración de Swagger
│   ├── middleware/         # Middleware para la aplicación
│   │   ├── authMiddleware.js      # Middleware de autenticación
│   │   ├── adminMiddleware.js     # Middleware para verificar roles de administrador
│   │   ├── errorMiddleware.js     # Middleware de manejo de errores
│   │   └── validators.js y middleware de validación # Middleware de validación
│   ├── utils/              # Utilidades y funciones auxiliares
│   │   ├── jwtUtils.js           # Utilidades para manejar JWT
│   │   ├── emailUtils.js         # Utilidades para enviar correos electrónicos
│   │   ├── smsUtils.js           # Utilidades para enviar SMS
│   │   └── logger.js             # Utilidad para el manejo de logs
│   ├── app.js              # Configuración principal de la aplicación
│   └── server.js           # Punto de entrada del servidor
├── tests/                  # Pruebas unitarias y de integración
│   ├── controllers/        # Pruebas para los controladores
│   ├── models/             # Pruebas para los modelos
│   ├── routes/             # Pruebas para las rutas
│   └── testUtils.js        # Utilidades para las pruebas
├── uploads/                # Carpeta para almacenar archivos subidos
```

```
├── swagger.yaml            # Documentación de la API con Swagger
├── package.json            # Dependencias y scripts del proyecto
├── .env                    # Variables de entorno
├── .env.ejemplo                    # Variables de entorno de ejemplo
└── README.md               # Documentación del proyecto
```

Requisitos del Backend

1. Autenticación y Autorización

- Los usuarios deben poder registrarse, iniciar sesión y recuperar su contraseña.
- Las rutas protegidas solo deben ser accesibles para usuarios autenticados.

2. Gestión de Productos

- Los usuarios gestores deben poder añadir, editar y eliminar productos.
- Los usuarios deben poder ver los productos disponibles y añadirlos a su cesta.

3. Gestión de Pedidos

- Los usuarios gestores deben poder abrir y cerrar pedidos.
- Los usuarios deben poder ver los pedidos abiertos y añadir productos a ellos.

4. Historial de Pedidos

- Los usuarios deben poder ver un historial de sus pedidos realizados.

5. Configuración de Cuenta

- Los usuarios deben poder actualizar su información personal y cambiar su contraseña.

6. Notificaciones

- Los usuarios deben recibir notificaciones sobre fechas límite de pedidos y actualizaciones.

7. Documentación de la API

- La API debe estar documentada utilizando Swagger para facilitar su uso y comprensión.

Ejemplo de Rutas de la API

Test

- **GET** `/api/test`: Prueba la conexión con la API y la base de datos.

Usuarios

- **POST** `/api/usuarios/registrar`: Registra un nuevo usuario.
- **POST** `/api/usuarios/login`: Autentica un usuario (login).
- **POST** `/api/usuarios/recuperar-password`: Envía un enlace para recuperar la contraseña.
- **GET** `/api/usuarios/obtenerTodos`: Obtiene todos los usuarios (solo administradores).
- **GET** `/api/usuarios/obtener/{id}`: Obtiene un usuario por su ID.
- **POST** `/api/usuarios/registrar`: Registra un nuevo usuario.
- **PATCH** `/api/usuarios/activar/{id}`: Activa o desactiva un usuario (solo administradores).
- **POST** `/api/usuarios/login`: Autentica un usuario (login).
- **POST** `/api/usuarios/recuperar-password`: Envía un enlace para recuperar la contraseña.
- **PUT** `/api/usuarios/actualizar/{id}`: Actualiza un usuario.
- **DELETE** `/api/usuarios/eliminar/{id}`: Elimina un usuario (solo administradores).
- **GET** `/api/usuarios/obtenerSaldo/{id_usuario}`: Obtiene el saldo de un usuario.

Productos

- **POST** `/api/productos/crear`: Crea un nuevo producto.
- **GET** `/api/productos/obtenerTodos`: Obtiene todos los productos.
- **GET** `/api/productos/obtener/{id}`: Obtiene un producto por su ID.
- **PUT** `/api/productos/actualizar/{id}`: Actualiza un producto.
- **DELETE** `/api/productos/eliminar/{id}`: Elimina un producto.
- **PATCH** `/api/productos/cambiarEstadoActivo/{id}`: Cambia el estado activo de un producto.

Proveedores

- **POST** `/api/proveedores/crear`: Crea un nuevo proveedor.
- **GET** `/api/proveedores/obtenerTodos`: Obtiene todos los proveedores.
- **GET** `/api/proveedores/obtener/{id}`: Obtiene un proveedor por su ID.
- **PATCH** `/api/proveedores/cambiarEstadoActivo/{id}`: Cambia el estado activo de un proveedor.

Pedidos

- **POST** `/api/pedidos/crear`: Crea un nuevo pedido.
- **GET** `/api/pedidos/obtenerTodos`: Obtiene todos los pedidos.
- **GET** `/api/pedidos/obtener/{id}`: Obtiene un pedido por su ID.
- **PUT** `/api/pedidos/actualizar/{id}`: Actualiza un pedido.
- **DELETE** `/api/pedidos/eliminar/{id}`: Elimina un pedido.
- **PATCH** `/api/pedidos/cambiarEstado/{id}`: Cambia el estado de un pedido.
- **PATCH** `/api/pedidos/cambiarEstadoActivo/{id}`: Cambia el estado activo de un pedido.

Detalle de Pedido

- **POST** `/api/detalle-pedido/crear`: Crea un nuevo detalle de pedido.
- **GET** `/api/detalle-pedido/pedido/{id_pedido}`: Obtiene todos los detalles de un pedido.
- **PUT** `/api/detalle-pedido/actualizar/{id}`: Actualiza un detalle de pedido.
- **DELETE** `/api/detalle-pedido/eliminar/{id}`: Elimina un detalle de pedido.

Familia-Proveedor

- **POST** `/api/familia-proveedor/crear`: Crea una nueva relación entre usuario y proveedor.
- **GET** `/api/familia-proveedor/obtenerProveedoresUsuario/{id_usuario}`: Obtiene todas las relaciones de un usuario.
- **GET** `/api/familia-proveedor/obtenerUsuariosProveedor/{id_proveedor}`: Obtiene todas las relaciones de un proveedor.
- **DELETE** `/api/familia-proveedor/eliminar/{id_usuario}/{id_proveedor}`: Elimina una relación entre usuario y proveedor.

Pedidos Periódicos

- **POST** `/api/pedido-periodico/crear`: Crea un nuevo pedido periódico.
- **GET** `/api/pedido-periodico/obtenerTodos`: Obtiene todos los pedidos periódicos.
- **GET** `/api/pedido-periodico/obtenerTodos/usuario/{id_usuario}`: Obtiene los pedidos periódicos de un usuario.

- **PUT** `/api/pedido-periodico/actualizar/{id}`: Actualiza un pedido periódico.
- **DELETE** `/api/pedido-periodico/eliminar/{id}`: Elimina un pedido periódico.
- **PATCH** `/api/pedido-periodico/cambiarEstadoActivo/{id}`: Cambia el estado activo de un pedido periódico.

Pagos

- **POST** `/api/pagos/crear`: Crea un nuevo pago.
- **GET** `/api/pagos/obtenerTodos`: Obtiene todos los pagos.
- **GET** `/api/pagos/pendientes-deudor/{id_usuario_deudor}`: Obtiene los pagos pendientes de un usuario deudor.
- **GET** `/api/pagos/pendientes-creditor/{id_usuario_creditor}`: Obtiene los pagos pendientes de un usuario acreedor.
- **PUT** `/api/pagos/cambiar-estado/{id}`: Cambia el estado de un pago.

Notificaciones

- **POST** `/api/notificaciones/enviar`: Envía una notificación.
- **PUT** `/api/notificaciones/marcar-leida/{id}`**: Marca una notificación como leída.**

Frontend Web VUE.js

Descripción del Frontend

El backend de la aplicación está desarrollado con **Node.js** y **VUE**. Se conecta al backend REST API para gestionar usuarios, productos, proveedores y pedidos. La aplicación está diseñada para ser escalable y segura.

Estructura del Proyecto

El frontend está estructurado de la siguiente manera:

```
frontend/
├── src/
│   ├── assets/
│   │   ├── styles/
│   │   │   ├── components.css
│   │   │   ├── global.css
│   │   │   └── variables.css
│   │   ├── cestakonsumo.png
│   │   └── cestaesparza.png
│   ├── components/
│   │   ├── FooterBar.vue
│   │   └── NavBar.vue
```

```
│   ├── directives/
│   │   └── autoblur.js
│   ├── services/
│   │   └── api.js
│   ├── store/
│   │   ├── alertStore.js
│   │   ├── alertStore.test.js
│   │   ├── index.js
│   │   └── index.test.js
│   ├── views/
│   │   ├── ComprasPage.vue
│   │   ├── ConfiguracionPage.vue
│   │   ├── DashboardPage.vue
│   │   ├── DetallesPedidoPage.vue
│   │   ├── GestionNotificacionesPage.vue
│   │   ├── GestionPagosPage.vue
│   │   ├── GestionPedidosPage.vue
│   │   ├── GestionPedidosPeriodicosPage.vue
│   │   ├── GestionProductosPage.vue
│   │   ├── GestionProveedoresPage.vue
│   │   ├── GestionSaldosPage.vue
│   │   ├── GestionUsuariosPage.vue
│   │   ├── HistorialPage.vue
│   │   ├── HomePage.vue
│   │   ├── LoginPage.vue
│   │   ├── RecuperarPasswordPage.vue
│   │   ├── RegistrarPage.vue
│   │   └── SoportePage.vue
│   ├── App.vue
│   └── main.js
├── public/
│   └── favicon.ico
├── index.html
├── package.json
├── vite.config.mjs
├── eslint.config.mjs
├── jsconfig.json
├── .env.development
├── .env.production
└── .gitignore
```

Diseño Visual

Guía de Estilo

Paleta de Colores

- **Color Primario**: `#4CAF50` (Verde) - Representa la sostenibilidad y la ecología.
- **Color Secundario**: `#FF9800` (Naranja) - Para botones y elementos destacados.
- **Color de Fondo**: `#FFFFFF` (Blanco) - Para una apariencia limpia y moderna.
- **Color de Texto**: `#333333` (Gris oscuro) - Para asegurar buena legibilidad.
- **Color de Enlaces**: `#007BFF` (Azul) - Para enlaces y acciones interactivas.
- **Color de Error**: `#DC3545` (Rojo) - Para mensajes de error y advertencias.

Tipografía

- **Fuente Principal**:
  - **Nombre**: `Open Sans`
  - **Estilo**: Regular

    - **Tamaño**: `16px` para texto normal, `24px` para encabezados.
- **Fuente Secundaria**:
    - **Nombre**: `Roboto`
    - **Estilo**: Italic
    - **Tamaño**: `14px` para subtítulos y notas.

Iconografía

- Utiliza iconos de **Font Awesome** o **Material Icons** para mantener la coherencia visual.
- Hay que verificar que los iconos sean simples y fáciles de entender.

Elementos de Interfaz

Botones

- **Estilo**:
    - Bordes redondeados (`5px` de radio).
    - Sombra para dar profundidad (`box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1)`).
- **Colores**:
    - Fondo: Color secundario (`#FF9800`).
    - Texto: Blanco (`#FFFFFF`).
- **Efecto Hover**:
    - Cambiar a un tono más oscuro del color secundario (`#E68900`).

Formularios

- **Estilo de Campos**:
    - Bordes redondeados (`5px` de radio).
    - Sombra ligera para destacar (`box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1)`).
- **Colores**:
    - Fondo: Blanco (`#FFFFFF`).
    - Texto: Gris oscuro (`#333333`).
- **Efecto Focus**:
    - Resaltar el borde con el color primario (`#4CAF50`).

Tarjetas de Producto

- **Estilo**:
    - Bordes redondeados (`10px` de radio).
    - Sombra para dar un efecto de elevación (`box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1)`).

- **Colores**:
  - Fondo: Blanco (`#FFFFFF`).
  - Texto: Gris oscuro (`#333333`).
- **Elementos**:
  - Imagen del producto en la parte superior.
  - Nombre del producto y precio debajo de la imagen.

Prototipos de la Aplicación Web

1. Pantalla de Inicio

- **Elementos Clave**:
  - Logo del grupo de consumo.
  - Breve descripción del grupo y el proposito.
  - Botón de "Iniciar Sesión" o "Registrarse".

Interacciones:

- Al hacer clic en "Iniciar Sesión", se abre la Pantalla de Inicio de Sesión.
- Al hacer clic en "Registrarse", se abre la Pantalla de Registro.

2. Pantalla de Registro

- **Elementos Clave**:
  - Formulario de registro con campos para:
    - Nombre completo.
    - Correo electrónico.
    - Movil
    - Contraseña.
    - Confirmar contraseña.
  - Botón de "Registrarse".
  - Enlace a "Iniciar Sesión" si ya tiene una cuenta.

Interacciones:

- Al hacer clic en "Registrarse", se valida la información:
  - Si es correcta, se muestra un mensaje de éxito y se redirige a la Pantalla de Inicio de Sesión.
  - Si hay errores, se muestran mensajes de validación (por ejemplo, "El correo ya está registrado").

3. Pantalla de Inicio de Sesión

- **Elementos Clave**:
  - Formulario de inicio de sesión con campos para:
    - Móvil o correo.
    - Contraseña.
  - Botón de "Iniciar Sesión".
  - Enlace a "¿Olvidaste tu contraseña?".

Interacciones:

- Al hacer clic en "Iniciar Sesión", se valida la información:
  - Si es correcta, se redirige al Dashboard o resumen mensual.
  - Si hay errores, se muestra un mensaje de error (por ejemplo, "Credenciales incorrectas").
- Al hacer clic en "¿Olvidaste tu contraseña?", se abre la Pantalla de Recuperación de Contraseña.

4. Pantalla de Recuperar Contraseña

- **Elementos Clave**:
  - Formulario de recuperar contraseña con campos para:
    - Móvil o email.
  - Botón: [Enviar Instrucciones].

Interacciones:

- Al hacer clic en "Enviar Instrucciones", se envía un correo o SMS con un enlace para restablecer la contraseña.

5. Pantalla de Dashboard (Tablero)

- **Elementos Clave**:
  - Cesta mensual. Lista de productos pedidos y su estado.
  - Acceso rápido a "Adquirir productos".
  - Sección de "Productos Disponibles".
  - Notificaciones sobre pedidos abiertos (no periódicos).

Interacciones:

- Al hacer clic en "Adquirir productos", se abre la Pantalla de compras.
- Al hacer clic en "Ver Historial de Pedidos", se abre la Pantalla de Historial de Pedidos.

6. Pantalla de Compras

- **Elementos Clave**:
  - Agrupados por proveedor/grupo productos

- Tiene que diferenciarse bien los proveedor/grupo productos que esten abiertos(pedidos abiertos)
- Lista de productos con:
    - Nombre del producto.
    - Descripción.
    - Precio.
    - Botón de "Añadir a la cesta/pedido conjunto". Solo si el pedido está abierto.
- Filtros para buscar productos por categoría o proveedor.

Interacciones:

- Al hacer clic en "Añadir producto", se actualiza el carrito/cesta y se muestra un mensaje de confirmación.
- Al hacer clic en "Ver Cesta", se abre la Pantalla de la cesta o dashboard.

7. Pantalla de Historial de Pedidos

- **Elementos Clave**:
    - Usuario.
    - Si es admin, desplegable para seleccionar el usuario (en caso de querer utilizar la misma pantalla para que el administrador pueda gestionar pedidos. Igual no es la mejor opción)
    - Lista de pedidos anteriores con:
        - Fecha de entrega.
        - Importe.
        - Estado del pedido (entregado, repartido, pendiente, cancelado, en proceso).
        - Botón [Ver Detalles].
    - Opción para ver detalles de cada pedido.

8. Pantalla de Detalles de Pedidos

- **Elementos Clave**:
    - Usuario.
    - Lista de productos del pedido:
        - Usuario que lo ha comprado.
        - proveedor/grupo productos
        - Unidades.
        - Importe.

9. Pantalla de Gestión de Usuarios (para administradores)

- **Elementos Clave**:
    - Lista de usuarios con:
        - Nombre.

        - Correo electrónico.
        - Teléfono.
        - Estado (activo/inactivo).
        - Rol
        - Proveedor/es asignados
        - Botones para "Editar" o "Eliminar" usuarios.
    - Opción para añadir nuevos usuarios.

Interacciones:

- Al hacer clic en "Editar", se abre modal de Edición de Usuario.
- Al hacer clic en "Eliminar", se muestra un mensaje de confirmación y se elimina el usuario.

10. Pantalla de Configuración de Cuenta

- **Elementos Clave**:
    - USuario
    - Formulario para actualizar información del usuario:
        - Nombre.
        - Correo electrónico.
        - Móvil
        - Rol
        - Proveedor/es
        - Contraseña (opcional).
    - Botón de "Guardar Cambios".

Interacciones:

- Al hacer clic en "Actualizar Información", se valida la información y se muestra un mensaje de éxito.

11. Pantalla de Soporte y Ayuda

- **Elementos Clave**:
    - Sección de preguntas frecuentes (FAQ).
    - Formulario de contacto para enviar consultas.
    - Información de contacto del grupo.

Interacciones:

- Al hacer clic en "Enviar Consulta", se muestra un mensaje de confirmación.

12. Pantalla de Gestión de Proveedores

- **Elementos Clave**:
    - Admin tiene permiso total.
    - Usuarios que gestionan un proveedor, permiso de modificar.

- Lista de proveedores con:
    - Nombre proveedor/grupo productos
    - Contacto
    - Método de pago
    - Correo electrónico.
    - Teléfono.
    - Estado (activo/inactivo).
    - Botones para "Editar" o "Eliminar" proveedores.
    - Listado de productos.
- Opción para añadir nuevos proveedores.

Interacciones:

- Al hacer clic en añadir proveedor se abre un modal con un formulario para introducir los datos del proveedor.

13. Pantalla de Gestión de Productos

- **Elementos Clave**:
    - Admin y usuario gestor del proveedor permiso total.
    - Lista de productos con:
        - Nombre.
        - Descripción.
        - Precio.
        - Proveedor/grupo productos.
        - Imagen
        - Estado (activo/inactivo).
        - Botones para "Editar" o "Eliminar" productos.
    - Opción para añadir nuevos productos.

14. Pantalla de Gestión de Pedidos

- **Elementos Clave**:
    - Admin y usuario gestor del proveedor permiso total.
    - El usuario gestor del proveedor será el encargado de abrir pedidos.
    - Lista de pedidos con:
        - Id.
        - Proveedor/grupo productos
        - Fecha de entrega (si hay).
        - Fecha de apertura.
        - Fecha de cierre.
        - Importe total.
        - Ver detalles de pedido.
    - Opción para crear un nuevo pedido. Abre modal con:

- Fecha de apertura. Desde cuando los usuarios podrán pedir productos.
- Fecha de cierre. Hasta cuando podrán los usuarios pedir, modificar o borrar productos.
- Fecha aproximada de reparto.
- Periodicidad. Para programar el pedido para que se abra con los mismos parámetros cada X días.

15. Barra de navegación

- **Elementos Clave**:
  - Usuario
  - Cerrar sesión
  - Dashboard
  - Historial pedidos
  - COmpras
  - Gestión de usuarios (solo admin)
  - Configuracion de cuenta
  - Ayuda
  - Gestión de proveedores (solo admin o gestor de proveedor)
  - Gestión de productos (solo admin o gestor de proveedor)
  - Gestión de pedidos (solo admin o gestor de proveedor)

16. Pie de página

- **Elementos Clave**:
  - RRSS
  - Datos de la web
  -

Flujo de Usuario para la Aplicación Web

1. Gestión de Productos

1) **Flujo de Usuario**:

   c) **Añadir un Producto (al proveedor/grupo de productos)** (para administradores y el usuario responsable de ese productor/grupodeproductos):
      1) El administrador selecciona "Añadir Producto".
      2) Completa un formulario con la información del producto (nombre, descripción, precio, proveedor/grupodeproductos).
      3) Hace clic en "Guardar" para añadir el producto a la base de datos.

d) **Editar o Eliminar un Producto** (para administradores y el usuario responsable de ese grupodeproductos/proveedor):
   1) El administrador selecciona un producto de la lista.
   2) Elige "Editar" para modificar la información o "Eliminar" para quitar el producto.
   3) Confirma la acción.

---

2. Gestión de Compras

1) **Flujo de Usuario**:
   a) **Acceso a la sección de Productos**:
      1) El usuario inicia sesión en la aplicación.
      2) Navega al menú y selecciona "Productos".
   b) **Visualización de Productos**:
      1) Se muestra una lista de productos disponibles con detalles (nombre, descripción, precio, proveedor/grupodeproductos).
      2) El usuario selecciona un grupo de productos (proveedor).
      3) Si el pedido a ese proveedor está abierto, estarán activados los botones para añadir productos.
      4) El usuario selecciona productos y cantidades.
      5) El usuario hace clic en **"Añadir al Pedido"**.
      6) El usuario puede aplicar filtros para buscar productos específicos.
   c) **Pantalla de dashboard - Cesta de Compras Mensual**:
      1) La cesta se resetea mensualmente. Excepto productos no entregados.
      2) El usuario ve un listado de los productos pedidos ese mes.
      3) Al lado de cada producto se muestra el estado.
   d) **Notificación al Proveedor y usuario gestor**:
      1) Cuando llega la fecha límite (fecha cierre de pedido), se notifica al proveedor y al usuario encargado de ese proveedor.
      2) Los productos se marcan como **"En proceso"**.
   e) **Seguimiento del Pedido**:
      1) El usuario puede ver el estado del pedido en el **Dashboard** y en el **Historial de Pedidos**.
      2) Estados: En proceso, En reparto, Entregado, Repartido.
   f) **Entrega del Pedido**:
      1) El proveedor entrega los productos.
      2) El usuario confirma la entrega.
      3) El sistema actualiza el estado del pedido a **"Entregado"** .

      H) **Reparto del Pedido**:
      4) El usuario gestor del proveedor entrega los productos.

5) El usuario confirma reparto.
6) El sistema actualiza el estado del pedido a **"Repartido"**.

---

2. Gestión de Pedidos

1) **Flujo de Usuario**:
   a) **Acceso a la sección de Pedidos**:
      1) El usuario gestor del proveedor o el admin inicia sesión y selecciona "Pedidos" en el menú.
   b) **Crear un Nuevo Pedido (conjunto)** (para administradores y el usuario responsable de ese productor/proveedor):
      1) El usuario hace clic en "Abrir Pedido".
      2) Se muestra un modal para crear el pedido.
      3) El usuario selecciona la fecha de apertura, fecha de cierre, fecha aproximada de entrega y la posibilidad de hacerlo periódico.
   c) **Gestión de Pedidos Existentes (Historial de pedidos)** (para administradores y el usuario responsable de ese productor/proveedor):
      1) El usuario puede ver una lista de pedidos anteriores.
      2) Selecciona un pedido para ver detalles, duplicar o cancelar si está pendiente.
      3) En los detalles se muestra el listado de productos, cantidades, usuario que ha comprado, importe, etc.

---

3. Cálculos Financieros (para administradores)

1) **Flujo de Usuario**:
   a) **Acceso a la sección de Finanzas**:
      1) El usuario inicia sesión y selecciona "Finanzas" en el menú.
   b) **Visualización de Gastos Mensuales**:
      1) Se muestra un resumen de los gastos mensuales de las familias.
      2) El usuario puede ver detalles de cada pedido y el saldo actual de cada usuario.
   c) **Gestión de Deudas**:
      1) El usuario puede ver a quién debe dinero y el importe.
      2) Opción para marcar deudas como pagadas.

---

4. Gestión de Usuarios (para administradores)

1) **Flujo de Usuario**:
   a) **Acceso a la sección de Gestión de Usuarios**:
      1) El administrador inicia sesión y selecciona "Usuarios" en el menú.

b) **Visualización de Usuarios**:
   1) Se muestra una lista de usuarios con detalles (nombre, correo electrónico, estado, movil, activo, rol, proveedor/es).
   2) Opción para buscar usuarios específicos.

c) **Añadir un Nuevo Usuario**:
   1) El administrador hace clic en "Añadir Usuario".
   2) Completa un formulario con la información del nuevo usuario.
   3) Hace clic en "Guardar" para añadir el usuario.

d) **Editar o Eliminar un Usuario**:
   1) El administrador selecciona un usuario de la lista.
   2) Elige "Editar" para modificar la información o "Eliminar" para quitar el usuario.
   3) Confirma la acción.

---

5. Notificaciones y Recordatorios

1) **Flujo de Usuario**:
   a) **Configuración de Notificaciones**:
      1) El usuario accede a "Configuración de Cuenta".
      2) Selecciona "Notificaciones".
      3) Activa o desactiva las notificaciones para fechas de pedidos y vencimientos.
   b) **Recepción de Notificaciones**:
      1) El usuario recibe notificaciones por correo electrónico o en la aplicación sobre fechas límite de pedidos y actualizaciones.

---

6. Historial de Pedidos

1) **Flujo de Usuario**:
   a) **Acceso a la sección de Historial de Pedidos**:
      1) El usuario inicia sesión y selecciona "Historial de Pedidos" en el menú.
   b) **Visualización de Pedidos Anteriores**:
      1) Se muestra una lista de pedidos anteriores con detalles como:
         A) Fecha del pedido.
         B) Productos solicitados.
         C) Estado del pedido (completado, pendiente, cancelado).
   c) **Ver Detalles de un Pedido**:
      1) El usuario selecciona un pedido de la lista.
      2) Se muestra una vista detallada del pedido, incluyendo:
         A) Productos solicitados.
         B) Cantidades.

C) Total del pedido.
D) Estado del pedido.

d) **Opción para Repetir un Pedido**:
1) El usuario puede seleccionar "Repetir Pedido" para crear un nuevo pedido con los mismos productos.
2) Se redirige al flujo de creación de pedidos con los productos preseleccionados.

---

7. Configuración de Cuenta

1) **Flujo de Usuario**:
a) **Acceso a la sección de Configuración de Cuenta**:
1) El usuario inicia sesión y selecciona "Configuración de Cuenta" en el menú.
b) **Visualización de Información de la Cuenta**:
1) Se muestra la información actual del usuario, incluyendo:
A) Nombre.
B) Correo electrónico.
C) Opciones de seguridad.
D) Movil
E) Activo
F) Rol
G) Proveedor/es
c) **Actualizar Información**:
1) El usuario puede editar su nombre o correo electrónico.
2) Opción para cambiar la contraseña.
3) Hace clic en "Guardar Cambios" para aplicar las modificaciones.
d) **Cerrar Sesión**:
1) El usuario puede seleccionar "Cerrar Sesión" para salir de la aplicación.

---

8. Panel de Administración (para administradores)

1) **Flujo de Usuario**:
a) **Acceso al Panel de Administración**:
1) El administrador inicia sesión y selecciona "Panel de Administración" en el menú.
b) **Visualización de Resumen General**:
1) Se muestra un resumen de la actividad del grupo, incluyendo:
A) Total de usuarios.
B) Total de pedidos realizados.
C) Productos más solicitados.

c) **Gestión de Productos y Usuarios**:
   1) El administrador puede acceder a las secciones de "Gestión de Productos" y "Gestión de Usuarios" desde el panel.
   2) Opción para ver estadísticas y reportes sobre el consumo y pedidos.

d) **Configuración de Parámetros del Grupo**:
   1) El administrador puede ajustar configuraciones generales del grupo, como:
      A) Frecuencias de pedidos.
      B) Notificaciones automáticas.
   2) Hace clic en "Guardar Cambios" para aplicar las modificaciones.

---

9. Informes y Estadísticas

1) **Flujo de Usuario**:
   a) **Acceso a la sección de Informes**:
      1) El usuario o administrador inicia sesión y selecciona "Informes" en el menú.
   b) **Visualización de Informes Generales**:
      1) Se muestra una lista de informes disponibles, como:
         A) Consumo mensual.
         B) Gastos por familia.
         C) Productos más comprados.
   c) **Generación de Informes Personalizados**:
      1) El usuario puede seleccionar criterios para generar informes personalizados.
      2) Hace clic en "Generar Informe" para visualizar los datos.
   d) **Descarga de Informes**:
      1) Opción para descargar los informes en formato PDF o CSV.
      2) Se muestra un mensaje de confirmación de que el informe ha sido descargado.

Diagrama de Flujo de Pedidos

Este diagrama muestra el flujo de un pedido desde su creación hasta su entrega.

Pasos:

1. **Apertura de Pedido**: El usuario gestor abre un pedido.
2. **Compras/Añadir Productos**: Los usuarios añaden productos al pedido.
3. **Cierre de Pedido**: El usuario gestor cierra el pedido.
4. **Notificación al Proveedor**: El sistema notifica al proveedor.
5. **Entrega de Productos**: El proveedor entrega los productos.
6. **Reparto de Productos**: Los productos se reparten entre los usuarios.

Diagrama:

```
[Apertura de Pedido] -> [Añadir Productos] -> [Cierre de Pedido] ->
[Notificación al Proveedor] -> [Entrega de Productos] -> [Reparto de Productos]
```

Diagrama de Navegación

Este diagrama muestra la estructura de las páginas de la aplicación y cómo se navega entre ellas.

Páginas Principales:

- **Inicio**: Página de bienvenida.
- **Inicio de Sesión**: Página para iniciar sesión.
- **Registro**: Página para registrarse.
- **Recuperar contraseña**: Página para recuperar la contraseña.
- **Dashboard**: Página principal del usuario.
- **Productos**: Página de gestión de productos.
- **Proveedores**: Página de gestión de proveedores.
- **Pedidos**: Página de gestión de pedidos.
- **Historial**: Página de historial de pedidos.
- **Configuración**: Página de configuración de cuenta.
- **Soporte**: Página de soporte y ayuda.
- **Compras**: Página donde el usuario puede comprar productos (añadirlos al pedido abierto) o ver los productos de proveedores que no tienen pedido abierto.

Diagrama de Secuencia de Autenticación

Este diagrama muestra el flujo de autenticación de un usuario.

Pasos:

1. **Solicitud de Inicio de Sesión**: El usuario introduce sus credenciales.
2. **Validación de Credenciales**: El backend valida las credenciales.
3. **Generación de Token**: El backend genera un token JWT.
4. **Respuesta al Frontend**: El backend devuelve el token al frontend.
5. **Almacenamiento del Token**: El frontend almacena el token en el localStorage.
6. **Acceso a Rutas Protegidas**: El frontend usa el token para acceder a rutas protegidas.

Diagrama:

```
[Usuario] -> [Frontend] -> [Backend] -> [Base de Datos]
[Backend] -> [Frontend] -> [Usuario]
```

Diagrama de Componentes reutilizables del Frontend

Este diagrama muestra los componentes reutilizables del frontend y cómo interactúan entre sí.

Componentes:

- **NavBar**: Barra de navegación superior. Se muestra en toda la aplicación.
- **Footer**: Pie de página. Se muestra en toda la aplicación.
- **ProductCard**: Tarjeta de producto.
- **PedidoCard**: Tarjeta de pedido.
- **Notification**: Componente de notificaciones.

Diagrama:

NavBar

[Todas las páginas]

Footer

Entorno de Producción

Clouding.io

- **Descripción:** VPS contratado para alojar el proyecto.
- **Especificaciones del Servidor:**
  - Sistema Operativo: - ~~12~~ - 13
  - Capacidad: 10 GB
  - CPU: 0.5 vCores
  - Uso: Alojamiento del backend y frontend del proyecto.

Scripts para Instalar el Entorno y Arrancar el Proyecto

- **Instalación del Entorno:**
  - He creado un script (`instalar-backend-frontend-vps.sh`) que automatiza la instalación de dependencias necesarias como Node.js, PostgreSQL, PM2 y Nginx.
  - Configura el backend y frontend en sus respectivos directorios (`/var/www/daw-proyecto/backend` y `/var/www/daw-proyecto/frontend`).
  - Configura Nginx para servir el frontend y hacer proxy inverso al backend.
- **Arranque del Proyecto:**
  - El script (`reiniciar_frontend_backend_vps.sh`) permite reiniciar el backend y frontend en caso de actualizaciones o reinicios del servidor.
  - Utiliza PM2 para gestionar el backend y Nginx para servir el frontend.

DuckDNS.org

- **Descripción:** Servidor DNS dinámico gratuito utilizado para asignar un nombre de dominio al proyecto.
- **Configuración:**
  - Dominio: `ekonsumo.duckdns.org`
  - Acceso al proyecto:
    - **Backend:** https://ekonsumo.duckdns.org/api-docs/
    - **Frontend:** https://ekonsumo.duckdns.org/

Subida o Actualización de Código

- **Descripción:** He creado scripts para actualizar únicamente el código fuente del proyecto en el servidor.
- **Funcionamiento:**
  - Utiliza ~~SCP~~ para copiar los archivos de código fuente desde el entorno local al servidor.
  - Los scripts (`subir_codigo_fuente.sh`) aseguran que solo se transfieran los archivos necesarios, como controladores, modelos, rutas y configuraciones.
  - ~~Configura automáticamente los permisos en el servidor después de la transferencia.~~
  - Hay que recargar el servidor con reload_ekonsumo.sh

Seguridad

- **HTTPS:** Configurado para garantizar comunicaciones seguras entre el cliente y el servidor.

- **Firewall:** Configurado con `ufw` para permitir únicamente los puertos necesarios (80, 443, 22 y 3000).
- **Autenticación:** Uso de JWT para proteger las rutas del backend.
- **CORS:** Configurado para permitir únicamente solicitudes desde el dominio del frontend.

Copias de Seguridad

- **Base de Datos:**
  - Uso de `pg_dump` para realizar copias de seguridad periódicas de la base de datos PostgreSQL.
  - Las copias se almacenan en un directorio seguro del servidor y se transfieren a un almacenamiento en otro equipo
- **Código Fuente:**
  - Uso de scripts para realizar copias de seguridad del código fuente y configuraciones críticas.

Monitoreo

- **Herramientas Utilizadas:**
  - `PM2`: Para monitorear el estado del backend.
  - `fail2ban`: Para bloquear intentos de acceso no autorizados.
  - Logs de Nginx: Configurados para registrar accesos y detectar actividades sospechosas.

Jobs Programados (Cron)

El sistema utiliza jobs programados mediante cron para automatizar tareas periódicas:

- Liquidación mensual (ekonsumo-liquidacion-mensual): cron '10 2 1 * *' (día 1 de cada mes a las 02:10). Genera las liquidaciones de pago entre usuarios.
- Pedidos periódicos (ekonsumo-pedidos-periodicos): cron '0 2 * * *' (cada día a las 02:00). Lee las configuraciones de Pedido_Periodico y genera pedidos automáticamente según la periodicidad configurada (días entre pedidos, día de apertura, cierre y entrega). Requiere la tabla Pedido_Periodico_Generacion.

Tabla Pedido_Periodico_Generacion: almacena la fecha de última generación para cada configuración periódica, evitando duplicados.
Scripts en servidor: /usr/local/bin/ekonsumo-liquidacion-mensual y /usr/local/bin/ekonsumo-pedidos-periodicos
Logs: /var/www/daw-proyecto/logs/